

Falco runtime detection rules

Detecting suspicious behaviour on a Linux host, and tuning the noise

Administration Unix: midterm project

Tony Doumit | Supervised by Prof. Fadi Chebli | October 2026

Falco watches what programs do

Overview

1. A host-based intrusion detection system (IDS)
2. Reads the system calls that programs make to the kernel
3. Compares each one with a set of rules, and alerts on a match

Falco never blocks anything.

Falco: detects

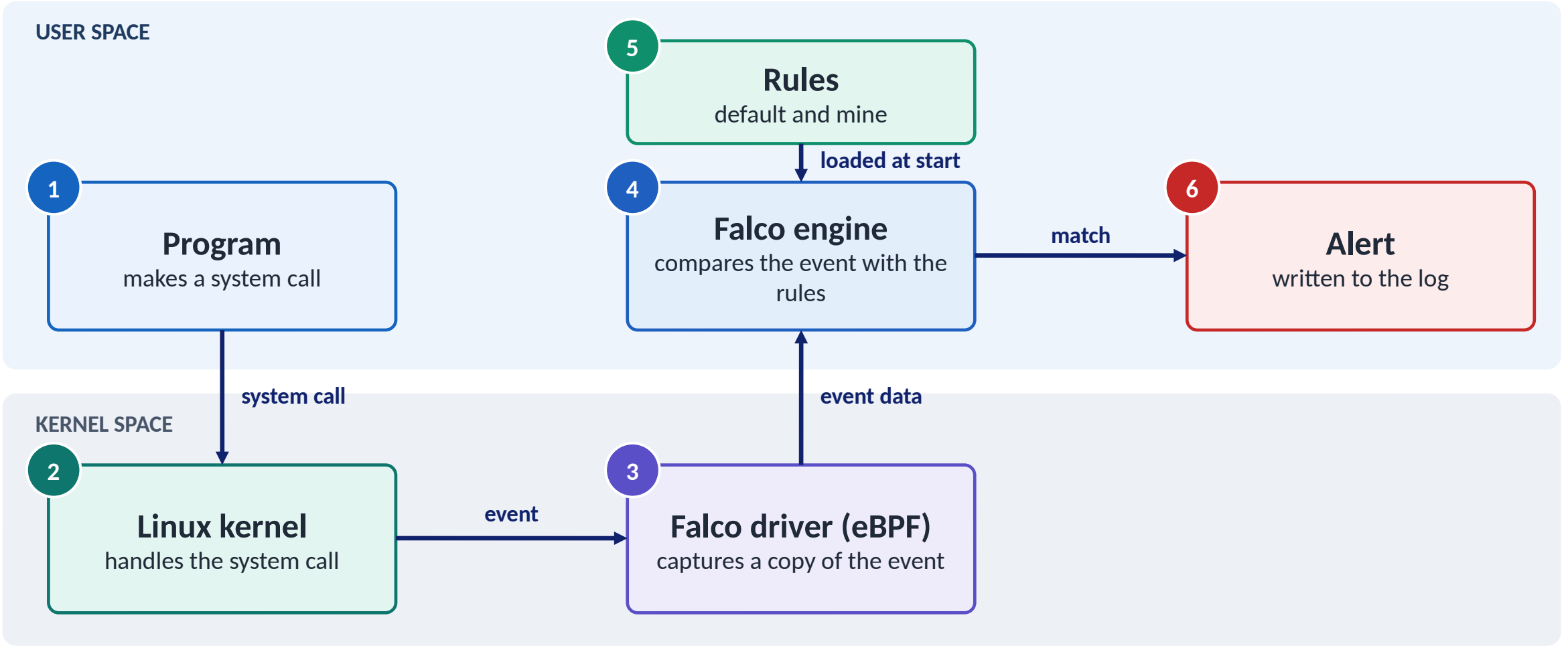
Works on a copy of the event. The alert comes after the action.

An IPS: prevents

Sits in the path of the action and can block it.

How Falco works

Architecture



The kernel does not wait for Falco: the driver captures a copy of the event, so nothing is blocked.

The lab, and one lesson

Lab

1. **A virtual machine**
   Ubuntu Linux in VirtualBox, isolated from my laptop

2. **Falco installed on it**
   watching the machine from the inside

3. **Snapshots**
   a way back after any mistake

Check the versions first

Older Falco
reported to stop on this new kernel

SO

Newer Falco
has the fix, so I installed it

Lesson: check compatibility before installing a tool that runs in the kernel.

The method I used for every rule

Method

1. **Get the alert**
2. **Find the rule**
3. **Read the rule**
4. **Measure the noise**
5. **Verify the program**
6. **Tune**
7. **Validate and restart**
8. **Measure again, with a control**

A rule that works is not finished: it also has to be quiet.

Default rule 1: read sensitive file

Credentials

What it does

Alerts when a program that is not trusted reads a sensitive file, like /etc/shadow or the login files in /etc/pam.d.

Warning

27 false alerts in one boot, all from the login manager

```
Warning Sensitive file opened for reading by non-trusted program
file=/etc/pam.d/gdm-password
process=gdm-session-worker
proc_exepath=/usr/libexec/gdm-session-worker
user=root  user_loginuid=1000
terminal=0   (no person typed it)
```

A real alert from my virtual machine.

From 27 false alerts to 0

Tuning

1 Verify

dpkg -S and dpkg -V: the file belongs to the gdm3 package and is unchanged

2 Tune

One narrow exception, by the exact program path. The default file stays untouched.

3 Prove

The count drops to 0, and a real read of the password file still alerts

27 → 0

alerts in the same boot and login

After the exception: the count is 0

```
$ grep -c "non-trusted program"   0
```

Control: sudo cat /etc/shadow still raises the alert

```
Warning file=/etc/shadow  process=cat
```

Default rules 2 and 3

Execution

Run shell untrusted

Notice

A shell starts below an application that never starts shells, like a web server.

I tested it with a copy of bash named lighttpd.

```
Notice Shell spawned by untrusted
binary
parent=lighttpd
process=sh
parent_exe=/tmp/lighttpd
```

True positive, no noise

Execution from /dev/shm

Warning

A program starts from the memory folder, a favourite place to hide a tool.

I tested it with a copy of echo run from /dev/shm.

```
Warning File execution detected
from /dev/shm
process=hello_shm
proc_exepath=/dev/shm/hello_shm
parent=bash
```

True positive, no noise

Four ideas of my own

My rules

A

The setuid bit

A file gets the power of its owner: a possible backdoor

Privilege

B

Network scanners

Someone looks for open ports and services on the host

Discovery

C

Blinding Falco

Edit its rules, move or delete them, or stop the service

Defense evasion

D

Attack tools

Password crackers and break-in toolkits start on the host

Attack tools

Priority ladder, same colors on every slide: Notice Warning Critical Alert

Rule A: the setuid bit

Privilege

A file with the setuid bit runs with the power of its owner. If the owner is root, it is a spare key.

1. Attacker gets root for a short time
2. Plants a root-owned file and sets the bit
3. Loses access, but keeps the key
4. **Falco alerts the moment the bit is set**

```
$ chmod u+s /tmp/suid_demo
Warning Setuid or setgid bit set
mode=...S_ISUID
process=chmod
```

Warning

Tuned: package installs set the bit too. An exception (a package manager above the command, and a system folder) removed that noise. A bit set in /tmp still alerts.

Rule B: scanners and the login id

Discovery

nmap and nc list open ports. The tier follows the login id: the person who really logged in.

Who starts the tool	user	login id
Me, no sudo	tony	1000
Me, with sudo	**root**	**1000**
A real root login	root	0
A service	any	-1

login id not 0 — Critical

login id is 0 — Warning

With sudo the user is root, but the login id is not 0: so it is not a root login.

With sudo

```
Critical Network scanner started by a non-root login
user=root  user_loginuid=1000
process=nmap
```

Simulated root login

```
Warning Network scanner started by a root login
user=root  user_loginuid=0
process=nmap
```

Rule C: blinding Falco

Defense evasion

Edit the files

C1: a refused write — Critical

C2: a real write — Warning

Replace or delete

C3: move or delete a file in /etc/falco — Warning

Stop the service

C4: systemctl stop or disable — Alert

If Falco is stopped, nothing is watched. So I watch Falco too.

C1: a normal user tries to write and is refused

```
Critical Denied write attempt on a Falco file
file=/etc/falco/falco_rules.local.yaml
evt_res=EACCES
user=tony  user_loginuid=1000
```

C4: a real stop. The alert was saved before Falco stopped.

```
Alert Falco service stopped or disabled
process=systemctl
proc_exepath=/usr/bin/systemctl
user=root  user_loginuid=1000
```

Rule D: attack tools

Attack tools

Cracking passwords

`john, hashcat, hydra, medusa, ncrack`

They turn password hashes back into passwords, or try many passwords on a login

Break-in toolkits

`msfconsole, msfvenom, sqlmap`

Ready-made tools to find and exploit weaknesses in systems

Two tiers, as in rule B

Ordinary account, sudo, or service

Alert

A real root login

Critical

Higher than rule B: nobody needs these tools on a normal machine.

A real tool, hydra -h (help only, nothing attacked): same alert as the stand-ins

```
Alert Attack tool started by a non-root login
process=hydra  proc_exepath=/usr/bin/hydra  command=hydra -h
```

Limits I found by testing

Limits

1

Rules cannot count

A real port scan is many events, and Falco sees one at a time

2

A root process can change its login id

An attacker with root can lower the alert

3

Admin or attacker?

With sudo, they look the same to the file rules

Finding the limits is part of the work.

What I take away

12

rules tested

3 default and 9 of mine, in 4 ideas

3

tunings, measured

27 to 0, package noise to 0, nano 3 to 1

3

main limits

shown here, more in the report

Falco is a good detector of behaviour and a poor judge of intent. Its value comes from rules that are fitted, tested with controls, and written with their limits.

Now, live

Demo

1. Read the password file (default rule 1)
2. Set the setuid bit on a file (A)
3. A scanner, with and without sudo (B)
4. A refused write to the Falco folder (C1)
5. Write and delete a file there (C2, C3)
6. Try to disable Falco (C4)
7. An attack tool, the real hydra (D)

Left terminal
Falco alerts, live

Right terminal
My commands, one scene at a time

Questions

Thank you

Backup: rule A tuning evidence

Backup

The same package reinstall, after the exception

```
tony@falco-lab:~$ sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep -c "Setuid or setgid bit set on a file
 | filename"
sudo apt install --reinstall -y fuse3
sleep 3
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep -c "Setuid or setgid bit set on a file | filename"
5
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 1 reinstalled, 0 to remove and 10 not upgraded.
Need to get 25.2 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://lb.archive.ubuntu.com/ubuntu noble/main amd64 fuse3 amd64 3.14.0-5build1 [25.2 kB]
Fetched 25.2 kB in 2s (15.8 kB/s)
(Reading database… 153006 files and directories currently installed.)
Preparing to unpack …/fuse3_3.14.0-5build1_amd64.deb…
Unpacking fuse3 (3.14.0-5build1) over (3.14.0-5build1)…
Setting up fuse3 (3.14.0-5build1)…
update-initramfs: deferring update (trigger activated)
Processing triggers for man-db (2.12.0-4build2)…
Processing triggers for initramfs-tools (0.142ubuntu25.8)…
update-initramfs: Generating /boot/initrd.img-7.0.0-34-generic
5
tony@falco-lab:~$
```

Before the exception, each reinstall raised an alert (the count went 3 to 4, then 4 to 5).

After it, the count stays at 5: the package manager above the command, plus a system folder, is no longer an alert.

A bit set in /tmp still alerts, even with a fake package manager above it.

Backup: evidence, default rules

Backup

Default rule 1: the first alert, the login manager

```
23:14:31.967774636: Warning Sensitive file opened for reading by non-trusted program | file=/etc/pam.d/gdm-password gpar
ent=gdm3 ggparent=systemd gggparent=<NA> evt_type=openat user=root user_uid=0 user_loginuid=1000 process=gdm-session-wor
 proc_exepath=/usr/libexec/gdm-session-worker parent=gdm-session-wor command=gdm-session-wor terminal=0 container_id=hos
t container_name=host container_image_repository= container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

After the exception: the count is 0

```
tony@falco-lab:~$ sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep -c "non-trusted program"
[sudo] password for tony:
0
tony@falco-lab:~$
```

Control: sudo cat /etc/shadow still alerts

```
10:53:23.473440421: Warning Sensitive file opened for reading by non-trusted program | file=/etc/shadow gparent=sudo ggp
arent=bash gggparent=gnome-terminal- evt_type=openat user=root user_uid=0 user_loginuid=1000 process=cat proc_exepath=/u
sr/bin/cat parent=sudo command=cat /etc/shadow terminal=34817 container_id=host container_name=host container_image_repo
sitory= container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

Default rule 2: a shell started below a process named lighttpd

```
12:46:36.136566801: Notice Shell spawned by untrusted binary | parent_exe=/tmp/lighttp
d parent_exepath=/tmp/lighttpd pcmdline=lighttpd -c sh -c \"echo hello from a shell\";
 echo done gparent=bash ggparent=gnome-terminal- aname[4]=systemd aname[5]=systemd ana
me[6]=<NA> aname[7]=<NA> evt_type=execve user=tony user_uid=1000 user_loginuid=1000 pr
ocess=sh proc_exepath=/usr/bin/dash parent=lighttpd command=sh -c echo hello from a sh
ell terminal=34816 exe_flags=0 container_id=host container_name=host container_image_r
epository= container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

Backup: evidence, rules A and B

Backup

Default rule 3: a program run from /dev/shm

```
21:07:07.855742446: Warning File execution detected from /dev/shm | evt_res=SUCCESS file=<NA> proc_cwd=/home/tony/ proc_
pcmdline=bash user_loginname=tony group_gid=1000 group_name=tony evt_type=execve user=tony user_uid=1000 user_loginuid=1
000 process=hello_shm proc_exepath=/dev/shm/hello_shm parent=bash command=hello_shm hello from /dev/shm terminal=34816 e
xe_flags=EXE_WRITABLE container_id=host container_name=host container_image_repository= container_image_tag= k8s_pod_nam
e=<NA> k8s_ns_name=<NA>
```

Rule A: chmod u+s on a file

```
12:10:19.837471163: Warning Setuid or setgid bit set on a file | filename=/tmp/suid_demo fd= mode=S_IROTH|S_IWGRP|S_IRGR
P|S_IWUSR|S_IRUSR|S_ISUID user=tony user_uid=1000 user_loginuid=1000 process=chmod proc_exepath=/usr/bin/chmod parent=ba
sh command=chmod u+s /tmp/suid_demo terminal=34816 container_id=host container_name=host container_image_repository= con
tainer_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

Rule B with sudo: Critical, login id 1000

```
20:01:36.053599653: Critical Network scanner started by a non-root login | user=root user_uid=0 user_loginuid=1000 proce
ss=nmap proc_exepath=/tmp/nmap parent=sudo gparent=sudo ggparent=bash gggparent=gnome-terminal- command=nmap terminal=34
817 exe_flags=EXE_WRITABLE container_id=host container_name=host container_image_repository= container_image_tag= k8s_po
d_name=<NA> k8s_ns_name=<NA>
```

Rule B, simulated root login: Warning, login id 0

```
21:07:28.950774037: Warning Network scanner started by a root login | user=root user_uid=0 user_loginuid=0 process=nmap
proc_exepath=/tmp/nmap parent=sudo gparent=sudo ggparent=bash gggparent=gnome-terminal- command=nmap terminal=34817 exe_
flags=EXE_WRITABLE container_id=host container_name=host container_image_repository= container_image_tag= k8s_pod_name=<
NA> k8s_ns_name=<NA>
```

Backup: evidence, rules C and D

Backup

Rule C1: a normal user is refused (Critical)

```
bash: /etc/falco/falco_rules.local.yaml: Permission denied
12:10:17.523733800: Critical Denied write attempt on a Falco file | file=/etc/falco/falco_rules.local.yaml evt_res=EACCE
S user=tony user_uid=1000 user_loginuid=1000 process=bash proc_exepath=/usr/bin/bash parent=gnome-terminal- gparent=syst
emd ggparent=systemd gggparent=<NA> command=bash terminal=34816 container_id=host container_name=host container_image_re
pository= container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

Rule C4: a real stop of Falco (Alert), saved before it stopped

```
active
10:53:51.871626214: Alert Falco service stopped or disabled | user=tony user_uid=1000 user_loginuid=1000 process=systemc
tl proc_exepath=/tmp/systemctl parent=bash gparent=gnome-terminal- ggparent=syst
10:55:56.930845417: Alert Falco service stopped or disabled | user=root user_uid=0 user_loginuid=1000 process=systemctl
proc_exepath=/usr/bin/systemctl parent=sudo gparent=sudo ggparent=bash gggparent
```

Rule D: the real tool hydra, help only

```
22:40:31.958807014: Alert Attack tool started by a non-root login | user=tony user_uid=1000 user_loginuid=1000 process=h
ydra proc_exepath=/usr/bin/hydra parent=bash gparent=gnome-terminal- ggparent=systemd gggparent=systemd command=hydra -h
 terminal=34816 container_id=host container_name=host container_image_repository= container_image_tag= k8s_pod_name=<NA>
 k8s_ns_name=<NA>
```